

Compilation and Program Analysis: parallelism

Ludovic Henrio

Master 1, ENS de Lyon et Dpt Info, Lyon1

2025-2026



- 1 Generalities on Parallelism
- 2 CCS: a classical communication calculus
- 3 A brief introduction to weak memory models
- 4 An introduction to Futures
- 5 Adding parallelism to Mini-while
- 6 Different approaches to implement languages

Why parallelism?

- ❶ To go faster
Massive amount of computation, sometimes massively parallel, sometimes with complex parallelisation patterns
- ❷ To handle large amount of data: big data-bases, consistency problems, synchronisation is crucial
- ❸ To handle problems that are by nature concurrent, from system interruption to online applications with several users/distributed data or decisions

Different forms of parallelism

Shared memory

principle: processes can write and read data in common memory spaces example: threads in most languages generally you need a form of locking to be able to write things correctly or something similar (can be basic mutex or more complex locking like Java serialize)

Message passing

principle: communication between thread by sending/receiving messages several communication patterns exist synchronous/asynchronous/different send and receive primitives, etc.

High-level programming models

Can mix shared data and message passing or provide a high-level paradigm that is different, generally safer.

Example: parallel skeletons like map-reduce, actors, ...

Discussion on: Parallel vs concurrent vs distributed

Objectives of this course

- Study and write semantics for parallel programs.
- The case of CCS
- A digression on weak memory models
- Study one particular construct, futures.
- A few general words on how to implement languages.

What semantics for parallel programs?

No big step semantics for parallelism . Denotational semantics difficult too because somehow big-step (see later).

Traditional approach: do small step semantics with interleaving between small steps if P does $P \rightarrow P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n$ and Q does $Q_1 \rightarrow Q_1 \rightarrow Q_2 \rightarrow \dots \rightarrow Q_n$ then $P||Q$ does the combination of the two. This is expressed by a simple rule:

$$\frac{P \rightarrow P'}{P||Q \rightarrow P'||Q}$$

Plus either the symmetric rule, or parallel composition is commutative and associative. This is most of the time sufficient but sometimes not enough, e.g. not directly adapted to weak memory models, no “true concurrency” of the form:

$$\frac{P \rightarrow P' \quad Q \rightarrow Q'}{P||Q \rightarrow P'||Q'}$$

- 1 Generalities on Parallelism
- 2 CCS: a classical communication calculus
- 3 A brief introduction to weak memory models
- 4 An introduction to Futures
- 5 Adding parallelism to Mini-while
- 6 Different approaches to implement languages

CCS syntax

- Channel names: $a, b, c, \dots$
- Co-names: $\bar{a}, \bar{b}, \bar{c}, \dots$ (complementary « $\bar{a} = a$ »)
- Silent action (unobservable): τ
- Actions: $\mu ::= a \mid \bar{a} \mid \tau$
- Processes:

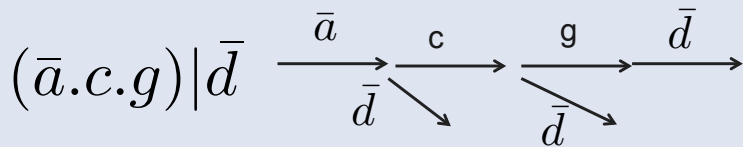
P, Q	$::=$	0	inaction
		$\mid \mu.P$	prefix
		$\mid P \mid Q$	parallel
		$\mid P + Q$	(external) choice
		$\mid (\nu a)P$	restriction
		$\mid \text{rec}_K P$	process P with definition $K = P$
		$\mid K$	(defined) process name

Intuitive Semantics from an “action” point of view

- $a.P$ offers action a and then becomes P
- $a.P + b.Q$ may offer either a and become P or b and become Q
- $(\nu a)P$ may offer any action of P , except a
- $P \mid Q$ may offer an action of P or of Q , but also if P offers a and Q offers a , they may synchronise (into a τ action)

We will use a labelled transition semantics
for CCS processes.

A Micro-example



CCS : behavioural semantics (1)

Operators and rules

- Action prefix:

$$\overline{\mu.P \xrightarrow{\mu} P}$$

- Communication:

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}$$

- Parallelism

$$\frac{P \xrightarrow{\mu} P'}{P|Q \xrightarrow{\mu} P'|Q}$$

$$\frac{Q \xrightarrow{\mu} Q'}{P|Q \xrightarrow{\mu} P|Q'}$$

CCS : behavioural semantics (2)

Operators and rules

- Non-deterministic choice

$$\frac{Q \xrightarrow{\mu} Q'}{P+Q \xrightarrow{\mu} Q'}$$

$$\frac{P \xrightarrow{\mu} P'}{P+Q \xrightarrow{\mu} P'}$$

- Scope restriction

$$\frac{P \xrightarrow{\mu} P' \quad \mu \neq a, \bar{a}}{(\nu a)P \xrightarrow{\mu} (\nu a)P'}$$

- Recursive definition

$$\frac{P[\text{rec}_K P / K] \xrightarrow{\mu} P'}{\text{rec}_K P \xrightarrow{\mu} P'}$$

Example

Apply semantic rules to infer one possible behaviour of:

$$\nu a.(a.b | (\bar{a}.c + \bar{a}.\bar{b}))$$

Are the following traces acceptable:

- $\tau.c.b$
- $a.\bar{a}.\tau$
- $\tau.\bar{b}.b$
- $\tau.\tau$

?

Notes on CCS and process algebras

- Different syntax exist, and plenty of variants, among them:
 - Different recursion: define process names
 - Guarded choice / guarded recursion
 - Passing data on channels ($\bar{a}(5) \mid a(x)$)
- More work on π -calculus than CCS. π -calculus is roughly CCS+sending channel names over channels.

Scopes and restrictions become more complex. You need something like:

$$((\nu x)(P|Q)) \equiv ((\nu x)P)|Q$$

- many research works on
 - equivalence relations for CCS/ π -calculus programs: Bisimulation and other equivalences.
 - typing the behaviour of programs using session types
 - security and probability aspects, etc.

Overall a vast topic

Cultural digression: About denotational semantics for parallelism

Recall: denotational semantics transforms a “program” into a “mathematical structure”
It looks like big-step but it depends on the structure generated. Generating something like a trace is possible too. Trace semantics exist but is not exactly denotational. However there are ways to get something more denotational, among our recent works:

- LAGC semantics with Reiner Hahnle, Einar Broch Johnsen et al.: kind of small-step denotational by “concretizing” the semantics at some points
- ctrees with Yannick Zakowski, Nicolas, Chappe, Steve Zdancewic (and others):
Generate a tree (coinductive) structure similar to a trace to take into account inputs, impure effects ... and non-determinism. Coq development for reasoning on languages and compilers.

- 1 Generalities on Parallelism
- 2 CCS: a classical communication calculus
- 3 A brief introduction to weak memory models
- 4 An introduction to Futures
- 5 Adding parallelism to Mini-while
- 6 Different approaches to implement languages

Weak memory models

Question: What are the possible results (value of c) for running these two threads in parallel (initially $a=b=c=0$)?

```
a=1;                b=1;
if (b==0) then      ||  if (a==0) then
    c++;             c++;
```

Answer: It depends ... a lot, there are many ways to interpret the program

A memory model gives the semantics of memory accesses

Memory model – SC

Sequential Consistency : Each thread executes in its order, only interleaving occurs.

The granularity of atomic instructions already plays a major role (e.g. in $y = x$ is there a temporary register?).

$$y = x \quad \parallel \quad \begin{array}{l} x = 1 \\ y = x \end{array}$$

If initially $x = y = 0$, at the end $y = 1$.

Memory model – TSO

Total Store Ordering : E.g. x86, Writings are not observed immediately by other threads but locally it is consistent. $\approx$ interleaving + local caches.

$$y = x \quad \parallel \quad \begin{array}{l} x = 1 \\ y = x \end{array}$$

If initially $x = y = 0$, at the end $y = 0$ or $y = 1$.

Different weak memory models

- The memory model of C, LLVM, ARM, etc. are more complex than TSO
- The hardware or the compiler may reorder memory operations according to rules. These rules typically state what are “independent read/writes”.
- Even single threaded programs are subject to re-ordering.
- Weak memory models allow for powerful optimisations but make programs difficult to verify/formalise/understand.
- Example of complex and expressive memory model: Promising, Pomsets with predicate transformers

Different weak memory models

- The memory model of C, LLVM, ARM, etc. are more complex than TSO
- The hardware or the compiler may reorder memory operations according to rules. These rules typically state what are “independent read/writes”.
- Even single threaded programs are subject to re-ordering.
- Weak memory models allow for powerful optimisations but make programs difficult to verify/formalise/understand.
- Example of complex and expressive memory model: Promising, Pomsets with predicate transformers

$$y = x \quad \left\| \begin{array}{l} z = y \\ x = 1 \end{array} \right.$$

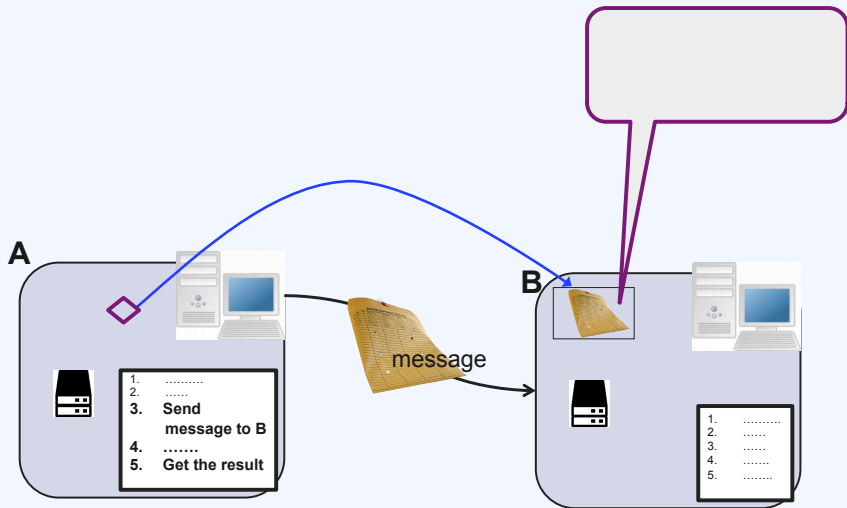
With promising, at the end, z can be 0 or 1. Promising can express the weak memory model of C++.

- 1 Generalities on Parallelism
- 2 CCS: a classical communication calculus
- 3 A brief introduction to weak memory models
- 4 An introduction to Futures
 - Principles
 - A case study from the literature: λ -calculus with futures
- 5 Adding parallelism to Mini-while
- 6 Different approaches to implement languages

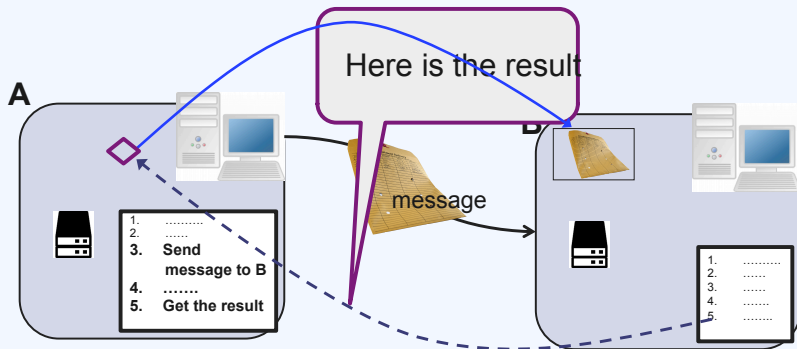
4 An introduction to Futures

- Principles
- A case study from the literature: λ -calculus with futures

Requests and replies



Requests and replies



4 An introduction to Futures

- Principles

- A case study from the literature: λ -calculus with futures

A simple λ -calculus with futures: Syntax

Terms: λ -calculus + futures:

$$e ::= (e \ e') \mid \lambda x. e \mid x \mid \text{get } e \mid f \mid \text{async}(e)$$

f appears during execution.

v is a value (fully evaluated term), i.e. $v ::= f \mid \lambda x. e$.

We could add other values, e.g. `int`.

A configuration consists of

- futures: $\text{fut}(f)$ (unresolved) or $\text{fut}(f \ v)$ (resolved with a value)
- and tasks ($\text{task}(f \ e)$).

References:

- A more complete lambda calculus with futures can be found in: *Niehren, Schwinghammer, Smolka. A Concurrent Lambda Calculus with Futures. TCS 2006,*
- simple lambda calculus with futures has been used in *Fernandez-Reyes, Clarke, Castegren, Vo, Forward to a Promising Future. Coordination 2018*

A simple λ -calculus with futures: Semantics

RED-LAMBDA

$$\text{task}(g \ E[(\lambda x.e) \ v]) \Rightarrow \text{task}(g \ E[e\{v/x\}])$$

RED-ASYNC

$$\frac{\text{fresh } f}{\text{task}(g \ E[\text{async}(e)]) \Rightarrow \text{fut}(f) \ \text{task}(f \ e) \ \text{task}(g \ E[f])}$$

CONTEXT

$$\frac{cn \Rightarrow cn'}{cn \ cn'' \Rightarrow cn' \ cn''}$$

END-TASK

$$\text{fut}(f) \ \text{task}(f \ v) \Rightarrow \text{fut}(f \ v)$$

RED-GET

$$\begin{aligned} \text{task}(f \ E[\text{get } f]) \text{fut}(f \ v) &\Rightarrow \\ \text{task}(f \ E[v]) \text{fut}(f \ v) & \end{aligned}$$

Note: configurations identified modulo reordering of tasks / futures, **What is E?**

Evaluation contexts

Evaluation contexts (sometimes called reduction contexts) used to focus on part of the configuration and reduce it. Compared to context rules they are more versatile: you can better choose what is in/out of the context.

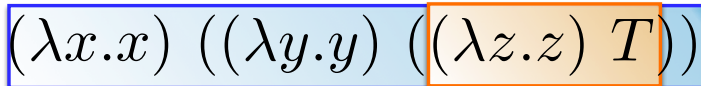
For lambda-calculus with futures:

$$E ::= E \ e \mid v \ E \mid \bullet \mid \text{get } E$$

This ensures call-by-value.

$$E[e] = E\{\bullet \leftarrow e\}$$

Reduction context



$$(\lambda x.x) \ ((\lambda y.y) \ ((\lambda z.z) \ T))$$

**Reduced
term**

Example of lambda-fut evaluation

Initial configuration:

A task containing the program to be evaluated: $task(f\ e)$ where e is the program and f is a future that will never be used.

What is the behaviour of $(\lambda x. get(async((\lambda y. y + y)\ x)))\ 3$?

- 1 Generalities on Parallelism
- 2 CCS: a classical communication calculus
- 3 A brief introduction to weak memory models
- 4 An introduction to Futures
- 5 Adding parallelism to Mini-while
 - Shared memory
 - Asynchronous function calls and futures
 - Typing futures in mini-while
 - Preservation: Principles
- 6 Different approaches to implement languages

5 Adding parallelism to Mini-while

- **Shared memory**
- Asynchronous function calls and futures
- Typing futures in mini-while
- Preservation: Principles

Mini-While Syntax (OLD) 1/2

Expressions:

$$e ::= c \mid e + e \mid e \times e \mid \dots$$

$$\mid x \quad \text{variable}$$

Statements:

$$S(Smt) ::= x := expr \quad \text{assign}$$

$$\mid x := f(e_1) \quad \text{simple function call}$$

$$\mid skip \quad \text{do nothing}$$

$$\mid S_1; S_2 \quad \text{sequence}$$

$$\mid \text{if } b \text{ then } S_1 \text{ else } S_2 \quad \text{test}$$

$$\mid \text{while } b \text{ do } S \text{ done} \quad \text{loop}$$

Mini-While Syntax (OLD) 2/2

Programs with function definitions and global variables

$Prog$	$::=$	$D \text{ FunDef } Body$	Program
$Body$	$::=$	$D; S$	Function/main body
D	$::=$	$var\ x : \tau \mid D; D$	Variable declaration
$FunDef$	$::=$	$\tau\ f(x_1 : \tau_1)\ Body; return\ e$	
		$\mid\ FunDef\ FunDef$	Function def

Structural Op. Semantics (SOS = small step) for mini-while (OLD – no fun)

$$(x := a, \sigma) \Rightarrow \sigma[x \mapsto Val(a, \sigma)]$$

$$(\text{skip}, \sigma) \Rightarrow \sigma$$

$$\frac{(S_1, \sigma) \Rightarrow \sigma'}{((S_1; S_2), \sigma) \Rightarrow (S_2, \sigma')} \quad \frac{(S_1, \sigma) \Rightarrow (S'_1, \sigma')}{((S_1; S_2), \sigma) \Rightarrow (S'_1; S_2, \sigma')}$$

$$\frac{Val(b, \sigma) = tt}{(\text{if } b \text{ then } S_1 \text{ else } S_2, \sigma) \Rightarrow (S_1, \sigma)}$$

$$\frac{Val(b, \sigma) = ff}{(\text{if } b \text{ then } S_1 \text{ else } S_2, \sigma) \Rightarrow (S_2, \sigma)}$$

Mini-while + shared memory (no functions)

Add parallel composition to statements

$$S ::= \dots | S || S'$$

And 2 reduction rules for parallelism:

PARALLEL1

$$\frac{(S_1, \sigma) \Rightarrow (S'_1, \sigma')}{(S_1 || S_2, \sigma) \Rightarrow (S'_1 || S_2, \sigma')}$$

PARALLEL2

$$\frac{(S_2, \sigma) \Rightarrow (S'_2, \sigma')}{(S_1 || S_2, \sigma) \Rightarrow (S_1 || S'_2, \sigma')}$$

And 2 special cases when a parallel task finishes:

ENDTASK1

$$\frac{(S_1, \sigma) \Rightarrow \sigma'}{(S_1 || S_2, \sigma) \Rightarrow (S_2, \sigma')}$$

ENDTASK2

$$\frac{(S_2, \sigma) \Rightarrow \sigma'}{(S_1 || S_2, \sigma) \Rightarrow (S_1, \sigma')}$$

Example mini-while shared memory

Compute the semantics of:

- $x := 0; (x := 2 \parallel \text{while } x < 3 \text{ do } x := x + 1 \text{ done})$

A final note: $\parallel$ is not often practical in the syntax one would prefer a spawn statement (different because spawn can create infinitely many tasks).

- 5 Adding parallelism to Mini-while
- Shared memory
 - **Asynchronous function calls and futures**
 - Typing futures in mini-while
 - Preservation: Principles

OLD SOS with functions (1/2)

Runtime configuration (Optional-Statement, Call-Stack, Stack, Store):

$$cn ::= (S, Ctx, \Sigma, sto) \mid (Ctx, \Sigma, sto)$$

$$(x := e, Ctx, \Sigma, sto) \Rightarrow (Ctx, \Sigma, sto[\Sigma(x) \mapsto Val(e, sto \circ \Sigma)])$$

$$\frac{(S_1, Ctx, \Sigma, sto) \Rightarrow (Ctx, \Sigma', sto')}{((S_1; S_2), Ctx, \Sigma, sto) \Rightarrow (S_2, Ctx, \Sigma', sto')}$$

$$\frac{(S_1, Ctx, \Sigma, sto) \Rightarrow (S'_1, Ctx, \Sigma', sto')}{((S_1; S_2), Ctx, \Sigma, sto) \Rightarrow (S'_1; S_2, Ctx, \Sigma', sto')}$$

+ rules for if, skip, and while

OLD SOS with functions (2/2)

CALL

$$\frac{bind_3(f, e_1..e_n, \Sigma, sto) = (S', \Sigma', sto')}{(x := f(e_1); S, Ctx, \Sigma, sto) \Rightarrow (S', (\Sigma, x := R(f); S) :: Ctx, \Sigma', sto')}$$

Ctx is a list of $(Stack, x := f(e_1); S)$. $R(f)$ is a marker that remembers the name of the function called

$bind_3(f, e_1, \Sigma, sto) = (S_f, \Sigma', sto[\ell_1 \mapsto v_1])$ if $body(f) = D_f; S_f$, $params(f) = [x_1]$,

$Vars(D_f) = \{y_1..y_k\}$

ℓ_1 fresh, $\ell'_1..\ell'_k$ fresh $Val(e_1, sto \circ \Sigma) = v_1$, $\Sigma' = \Sigma[x_1 \mapsto \ell_1, y_1 \mapsto \ell'_1..y_k \mapsto \ell'_k]$.

$$\frac{v = Val(ret(f), sto \circ \Sigma')}{((\Sigma, x := R(f); S) :: Ctx, \Sigma', sto) \Rightarrow (S, Ctx, \Sigma, sto[\Sigma(x) \mapsto v])}$$

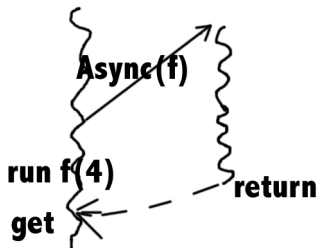
Futures: syntax and principles

Statements:

$S(Smt) ::=$	$x := expr$	assign
	$x := f(e_1)$	simple function call
	$x := Async(f(e_1))$	Asynchronous function call
	$x := get(e)$	future access (synchronisation)
	$skip$	do nothing
	$S_1; S_2$	sequence
	$if\ b\ then\ S_1\ else\ S_2$	test
	$while\ b\ do\ S\ done$	loop

Example (informally)

```
int f (int x) (  
  int z;  
  z:=x+x  
) return z  
  
(  
  int x,y;  
  fut<int> t;  
  t:=Async(f(3));  
  y:=f(4);  
  x:=get(t)  
)
```



Design choice: no global state

We have the choice between

- 1 Have a global state and allow race-condition between tasks. Versatile and looks like C threads. Drawback: data-races, what memory model?
- 2 Forget the global state that we had in function evaluation to have a more predictable semantics: no races between two tasks writing on the same memory.

The second case corresponds to MiniC where we have no global variable.

We specify the semantics for the second solution. To implement the first solution a global memory should be added to the configuration.

We thus suppose from now on that there is no global variable.

Future semantics for mini-while (1/4)

- Syntax: F, G range over future identifiers. Values (v) can be future identifiers.
- Configurations:

$$cn ::= (S, Ctx, \Sigma, sto)_F \mid (Ctx, \Sigma, sto)_F \mid fut(F, v) \mid cn \ cn'$$

Configurations are identified modulo reordering of tasks.

Note: compared to λ -calculus+fut we do not put unresolved futures in the configuration (we could).

- Sequential reductions: we can apply sequential rule in any task:

$$\frac{(S, Ctx, \Sigma, sto) \Rightarrow (S', Ctx', \Sigma', sto')}{(S, Ctx, \Sigma, sto)_F \ cn \Rightarrow (S', Ctx', \Sigma', sto')_F \ cn}$$

$$\frac{(S, Ctx, \Sigma, sto) \Rightarrow (Ctx', \Sigma', sto')}{(S, Ctx, \Sigma, sto)_F \ cn \Rightarrow (Ctx', \Sigma', sto')_F \ cn}$$

Future semantics for mini-while (2/4)

A naive solution for asynchronous function call:

ASYNC-CALL (BAD)

$$\frac{\text{bind}_3(f, e_1, \Sigma, sto) = (S', \Sigma', sto') \quad G \text{ fresh future}}{(x := \text{Async}(f(e_1)); S, Ctx, \Sigma, sto)_F \text{ cn} \Rightarrow (x := G; S, Ctx, \Sigma, sto)_F (S', \emptyset, \Sigma', sto')_G \text{ cn}}$$

Problem: we have lost the return expression that was somehow remembered by $R(f)$ in the synchronous call. We do not know what to fill the future G with.

Future semantics for mini-while (3/4)

A possible solution: add $return(e)$ to the valid Ctx and store the returned expression in the call-stack:

Ctx is a list of $(Stack, Stm)$ possibly with $return(e)$ as the last element of the list.

ASYNC-CALL

$$\frac{bind_3(f, e_1, \Sigma, sto) = (S', \Sigma', sto') \quad G \text{ fresh future}}{(x := Async(f(e_1)); S, Ctx, \Sigma, sto)_F \text{ } cn \Rightarrow (x := G; S, Ctx, \Sigma, sto)_F (S', return(ret(f)), \Sigma', sto')_G \text{ } cn}$$

Future semantics for mini-while (4/4)

End of function execution and future access:

FUT-RESOLVE

$$\frac{v = Val(e, sto \circ \Sigma)}{(return(e), \Sigma, sto)_F \text{ *cn*} \Rightarrow fut(F, v) \text{ *cn*}}$$

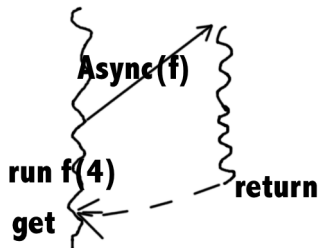
GET

$$\frac{G = Val(e, sto \circ \Sigma)}{(x := get(e); S, Ctx, \Sigma, sto)_F fut(G, v) \text{ *cn*} \Rightarrow (x := v; S, Ctx, \Sigma, sto)_F fut(G, v) \text{ *cn*}}$$

Example (semantics)

use the semantics to evaluate the previous example

```
int f (int x) (  
  int z;  
  z:=x+x  
) return z  
  
(  
  int x,y;  
  fut<int> t;  
  t:=Async(f(3));  
  y:=f(4);  
  x:=get(t)  
)
```



5 Adding parallelism to Mini-while

- Shared memory
- Asynchronous function calls and futures
- **Typing futures in mini-while**
- Preservation: Principles

Typing judgment for statements: reminder

$$\frac{\Gamma_D(D) \vdash S}{\emptyset \vdash D; S}$$

With $\Gamma_D(D) \triangleq \{x \mapsto \tau \text{ s.t. } x : \tau \in D\}$ (undef if multiple defs).

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}}$$

$$\Gamma \vdash \text{skip}$$

$$\Gamma \vdash x : \Gamma(x)$$

$$\frac{c \in \mathbf{Z}}{c : \text{int}}$$

$$\frac{b \in \mathbb{B}}{c : \text{bool}}$$

$$\frac{\Gamma \vdash S_1 \quad \Gamma \vdash S_2}{\Gamma \vdash S_1; S_2}$$

$$\frac{\Gamma \vdash x : \tau \quad \Gamma \vdash e : \tau}{\Gamma \vdash x := e}$$

$$\frac{\Gamma \vdash e : \text{bool} \quad \Gamma \vdash S_1 \quad \Gamma \vdash S_2}{\Gamma \vdash \text{if } e \text{ then } S_1 \text{ else } S_2}$$

$$\frac{\Gamma \vdash e : \text{bool} \quad \Gamma \vdash S}{\Gamma \vdash \text{while } e \text{ do } S \text{ done}}$$

Type function calls and typing program

To type a program we type all method bodies:

$$\frac{\Gamma_D(D_m), \Gamma_f(Fundef) \vdash S \quad \forall (\tau \ f(x_1 : \tau_1) \ S_f; \text{return } e) \in Fundef. \quad \Gamma_l \vdash e : \tau \wedge \Gamma_D(D) + \Gamma_l, \Gamma_f(Fundef) \vdash S_f \quad \text{with } \Gamma_l = \Gamma_D(x_1 : \tau_1; D_f)}{\vdash D \ Fundef \ D_m; S}$$

CALL

$$\frac{\Gamma_f(f) = \tau_1 \rightarrow \tau \quad \Gamma \vdash e : \tau_1 \quad \Gamma \vdash x : \tau}{\Gamma, \Gamma_f \vdash x := f(e)}$$

Note: recall there is no global variable in our current setting.

Adding future types

Previous type syntax:

$$\tau ::= int \mid bool$$

New types can be futures:

$$\tau ::= int \mid bool \mid fut < \tau >$$

Future types can be declared: `fut<int> x, y`

We check structural type equivalence.

Typing rules for futures

ASYNC

$$\frac{\Gamma_f(f) = \tau_1 \rightarrow \tau \quad \Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash x : fut < \tau >}{\Gamma, \Gamma_f \vdash x := \mathbf{Async}(f(e_1))}$$

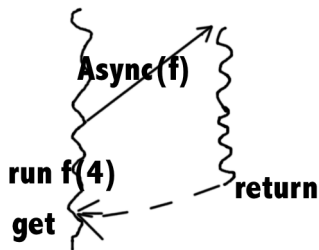
GET

$$\frac{\Gamma \vdash e : fut < \tau > \quad \Gamma \vdash x : \tau}{\Gamma, \Gamma_f \vdash x := \mathbf{get}(e)}$$

Example (type)

Type the previous example (skip the typing of f)

```
int f (int x) (  
  int z;  
  z:=x+x  
) return z  
  
(  
  int x,y;  
  fut<int> t;  
  t:=Async(f(3));  
  y:=f(4);  
  x:=get(t)  
)
```



- 5 Adding parallelism to Mini-while
 - Shared memory
 - Asynchronous function calls and futures
 - Typing futures in mini-while
 - **Preservation: Principles**

Preservation

Definition:

consider a well typed program $Prog$, a configuration cn reachable by executing $Prog$, we have

$$cn \Rightarrow cn' \wedge \Gamma_f, \mathbf{\Gamma}_{fut} \vdash cn \implies \exists \Gamma'_{fut}, \Gamma_f, \mathbf{\Gamma}'_{fut} \vdash cn'$$

What is a well-typed configuration? i.e. define the assertion $\Gamma_f, \mathbf{\Gamma}_{fut} \vdash cn$ What is $\mathbf{\Gamma}_{fut}$?

Definition (Configuration typing (very OLD))

$$\Gamma \vdash (S, \sigma) \iff (\Gamma \vdash S \wedge \forall x. \emptyset \vdash \sigma(x) : \tau \iff \Gamma(x) = \tau)$$

Now we have:

$$cn ::= (S, Ctx, \Sigma, sto)_F \mid (Ctx, \Sigma, sto)_F \mid fut(F, v) \mid cn \quad cn'$$

With Ctx of the form $(\Sigma, S) :: \dots :: (\Sigma_n, S_n) :: return(e)$ (no $return(e)$ for the main task).

Well-typed configuration

Suppose $\Gamma, \Gamma_f \vdash (S, \sigma)$ defined similarly to before (Γ_f added). We can define:

$$\frac{\Gamma, \Gamma_f \vdash (S, sto \circ \Sigma) \quad Ctx = (\Sigma_0, S_0) :: \dots :: (\Sigma_n, S_n) :: return(e) \quad \forall i \in [1..n]. \Gamma_i, \Gamma_f \vdash (S_i, sto \circ \Sigma_i) \quad \Gamma_n \vdash e : \Gamma_{fut}(F)}{\Gamma_f, \Gamma_{fut} \vdash (S, Ctx, \Sigma, sto)_F} \quad \frac{\emptyset \vdash v : \Gamma_{fut}(F)}{\Gamma_f, \Gamma_{fut} \vdash fut(F, v)}$$

$$\frac{\Gamma_f, \Gamma_{fut} \vdash cn \quad \Gamma_f, \Gamma_{fut} \vdash cn'}{\Gamma_f, \Gamma_{fut} \vdash cn \ cn'}$$

Problem (same as with functions): Γ , Γ_i are undefined. It is the typing environment of the function that contains the considered statement. We can for example annotate configurations with the name of the function that is currently evaluated and recover the typing environment (existence of a typing environment is sufficient).

Now: How to prove that small step semantics preserves well-typed configurations?

About Progress

State a progress property This entails absence of deadlock

Write a program that can deadlock

Alternatively, we can state a weaker progress property:

Any well-typed configuration that cannot progress is either a final configuration or exhibits a cycle of dependencies between futures: there is a list of future identifiers such that the task responsible for computing F_1 is performing a get on future F_2 , ... the task responsible for computing F_n is performing a get on future F_0 .

In other words: typing rules out all kinds of stuck configurations except cycles of futures.

- 1 Generalities on Parallelism
- 2 CCS: a classical communication calculus
- 3 A brief introduction to weak memory models
- 4 An introduction to Futures
- 5 Adding parallelism to Mini-while
- 6 Different approaches to implement languages

Different approaches to implement languages

Classical compilation: as seen in course

Source-to-source compilation: compiling to assembler is tedious and restricted to one architecture. Many source-to-source compilers, e.g. language-to-C / language-to-Java

Libraries: No translation at all. Relies on software engineering expertise to implement rich libraries DSLs while staying in the restriction of the host language

Approaches can be mixed.

Domain specific language (DSL) are programming languages with a high level of abstraction, designed/optimized for a specific class of problems. They are often implemented using source-to source compilation and libraries.

Next: 2 examples.

ProActive:

A Java **API + Tools** for Parallel, Distributed Computing

A uniform framework: **An Active Object pattern**

A formal model behind: **Determinism (POPL'04)**

- **Programming Model (Active Objects):**
- **Asynchronous Remote Invocations, Wait-By-Necessity**
- **Groups, Mobility, Components, Security, Fault-tolerance, Load balancing**
- **Environment:**
 - **XML Deployment Descriptors, File Transfers**
 - **Interfaced with:** `rsh`, `ssh`, `LSF`, `PBS`, `Globus`, `Jini`, `SUN Grid Engine`



Creating active objects

An object created with `A a = new A (obj, 7);`
can be turned into an active and remote object:

- **Object-based:**

```
a = (A) ProActive.turnActive (a, node);
```

- **Instantiation-based:**

```
A a = (A) ProActive.newActive («A», param, node);
```

The "node" is the AO container.

Remaining of the code unchanged → "Transparency"

Example 2: The Encore language approach

A language with objects, futures, actors, etc. with a rich type system to optimise data access while preventing data-races.

Many advanced features: Ownership types, parallel futures, forwarding, ...

Compiled into C (source-to-source compilation)

Relies on a specific C library, and an existing Actor library in C (pony) with a dedicated runtime (PonyRT) for final compilation and execution

Tiny code example (observe the dedicated syntax):

```
defrun() : void {
    let fut = service.provide()
        client =new Client()
in {
    client.send(get fut);
...
}
```

Today's course summary

- An introduction to CCS
- Brief introduction to weak memory models
- Brief introduction to different ways to implement languages
- A tour on how to add parallelism to mini-while